



Advanced Terraform on Azure

Lab 1 - Configuration and Identity Churn

Expected Duration	1
Teaching Points – What This Lab Is Designed to Teach	1
Before you begin	2
Phase 1 – Initial deployment	2
Phase 2 – No churn	5
Phase 3 – Configuration churn	6
Phase 4 – Identity churn.....	8
Phase 5 - For_each to the rescue ?	10
Phase 6 - Maps to the rescue !	12
Phase 7 – Clean up that mess before you leave !	13

Expected Duration

This lab should take 40-50 minutes to complete

Teaching Points – What This Lab Is Designed to Teach

This lab is deliberately not about writing “perfect” Terraform from the start. It is about understanding how Terraform state, resource identity, and configuration choices interact over time - and how seemingly small design decisions can lead to large operational consequences.

Across the phases in this lab, you will repeatedly observe Terraform behaving in ways that are *technically correct*, but operationally undesirable. That tension is intentional.

By the end of the lab, you should be able to clearly explain:

- Why Terraform sometimes proposes large numbers of changes even when your *intent* feels minimal

- The difference between **configuration churn** (unnecessary updates) and **identity churn** (forced replacement)
- Why list order, indexing, and derived values are dangerous foundations for long-lived infrastructure
- How Terraform decides *what* a resource is, not just *how* it is configured
- Why state-aware tools require more than “it works once” thinking

You will also see why certain Terraform features — such as `count`, `for_each`, `-target`, and `--parallelism` - are powerful but sharp tools. Used carelessly, they can introduce risk, race conditions, or drift. Used deliberately, they enable precise and predictable infrastructure management.

This lab progresses intentionally:

- Early phases **create instability** so you can observe churn
- Middle phases **explain why it happens**
- Final phases **remove the causes**, not just the symptoms

The goal is not simply to reach a “clean” end state, but to understand why that end state is stable — and why earlier designs were not.

Before you begin

1. Navigate to the `lab1/guided` folder in VSC
2. Move into `lab1/guided` directory at the terminal.

This will be your working directory and root module for this lab. Make all changes in this directory **ONLY**

The `lab1/original` directory contains a duplicate copy of `lab1/guided` content and is provided for recovery if needed.

Phase 1 – Initial deployment

3. In `c:\qatfadvez\labs1\guided`, update `terraform.tfvars` with your `subscription_id`
4. Run...

```
cd c:\qatfadvez\lab1\guided
az login
```

5. Review `main.tf`, `variables.tf` and `terraform.tfvars`

This simple code example creates a resource group in which there will be a Network Security Group (NSG) with Ingress rules.

Note that lines 18 to 32 of main.tf deploys the NSG rules using count

As shown in **terraform.tfvars**, there are 2 elements currently defined in the **allowed_cidrs** variable; 10.0.0.0/24 and 10.1.0.0/24, so terraform will create 2 NSG rule resources using the **count index** for addressing and for determining the rule priority.

Expected rules;

```
azurerm_network_security_rule.allow_https[0]
  name: allow-https-0
  priority: 200
  source_address_prefix = 10.0.0.0/24
azurerm_network_security_rule.allow_https[1]
  name: allow-https-1
  priority: 201
  source_address_prefix = 10.1.0.0/24
```

6. Deploy the resources;

```
terraform init
terraform apply --auto-approve
```

Note: If this is the first lab of the day then the initial plan/apply operations may 'hang'. If nothing happens after 1 minute or so, then perform this one-off fix and retry the plan/apply operation ...

Press **Ctrl+c** twice to 'break out'

Run the following command from you current working directory...

```
powershell -ExecutionPolicy Bypass -File C:\qatfadva\artifacts\fix\initfix.ps1
```

7. Verify the resources are created in state;

```
terraform state list
```

```
azurerm_network_security_group.app
azurerm_network_security_rule.allow_https[0]
azurerm_network_security_rule.allow_https[1]
azurerm_resource_group.rg
```

terraform state show azurerm_network_security_rule.allow_https[0]

```
PS C:\qatfadavaz\lab1\guided> terraform state show azurerm_network_security_rule.allow_https[0]
# azurerm_network_security_rule.allow_https[0]:
resource "azurerm_network_security_rule" "allow_https" {
  access              = "Allow"
  description         = null
  destination_address_prefix = "*"
  destination_port_range = null
  destination_port_ranges = [
    "443",
  ]
  direction           = "Inbound"
  id                  = "/subscriptions/f26494cf-0ac8-4c97-9ab8-bcb83a2e38a9/resourceps/landing-zone-dev-nsg/securityRules/allow-https-0"
  name                = "allow-https-0"
  network_security_group_name = "landing-zone-dev-nsg"
  priority             = 200
  protocol             = "Tcp"
  resource_group_name  = "landing-zone-dev-rg"
  source_address_prefix = "10.0.0.0/24"
  source_port_range    = "*"
}
```

terraform state show azurerm_network_security_rule.allow_https[1]

```
PS C:\qatfadvaz\lab1\guided> terraform state show azurerm_network_security_rule.allow_https[1]
# azurerm_network_security_rule.allow_https[1]:
resource "azurerm_network_security_rule" "allow_https" {
  access                = "Allow"
  description           = null
  destination_address_prefix = "*"
  destination_port_range = null
  destination_port_ranges = [
    "443",
  ]
  direction             = "Inbound"
  id                    = "/subscriptions/f26494cf-0ac8-4c97-9ab8-bcb83a2e38a9/resourceGroups/ps/landing-zone-dev-nsg/securityRules/allow-https-1"
  name                  = "allow-https-1"
  network_security_group_name = "landing-zone-dev-nsg"
  priority              = 201
  protocol              = "Tcp"
  resource_group_name   = "landing-zone-dev-rg"
  source_address_prefix = "10.1.0.0/24"
  source_port_range     = "*"
}
```

8. Navigate to the Portal to verify the rules exist. Note that the names are derived from the index position;

Priority ↑↓	Name ↑↓	Port ↑↓	Protocol ↑↓	Source ↑↓
▼ Inbound Security Rules				
200	allow-https-0	443	Tcp	10.0.0.0/24
201	allow-https-1	443	Tcp	10.1.0.0/24

Phase 2 – No churn

9. Comment line **6** and uncomment line **7** of terraform.tfvars (a new cidr has been inserted at the end of the list)
10. Plan the changes;

terraform plan

The plan is 1 to add, 0 to change.

The new resource will be azurerm_network_security_rule.allow_https[2]. This will have no effect on the existing resources [0] and [1], hence no changes and no churn.

11. Apply the changes;

terraform apply --auto-approve

Priority ↑↓	Name ↑↓	Port ↑↓	Protocol ↑↓	Source ↑↓
▼ Inbound Security Rules				
200	allow-https-0	443	Tcp	10.0.0.0/24
201	allow-https-1	443	Tcp	10.1.0.0/24
202	allow-https-2	443	Tcp	10.2.0.0/24

When using count, adding items to the end of a list produces predictable, stable behaviour. Terraform adds only the new resource and leaves existing ones untouched. However, this stability depends entirely on preserving the order of the list—insertions or deletions elsewhere will break it.

Phase 3 – Configuration churn

12. Comment line **7** and uncomment line **8** of terraform.tfvars (a new cidr has been inserted at the start of the list)

13. Plan the changes

terraform plan

The plan is 1 to add, 3 to change.

What is happening ?

Scroll up and review the plan...

The '1 to add' is **not** a new rule[0] referencing 192.168.1.0/24.

Rule[0] already exists in state, but it is currently referencing 10.0.0.0/24. So, one of the 3 changes is to change its current source_address_prefix from 10.0.0.0/24 to 192.168.1.0/24 because this cidr is now first on the list. Its priority will be 200, again because it is first on the list...

```
# azurerm_network_security_rule.allow_https[0] will be updated in-place
~ resource "azurerm_network_security_rule" "allow_https" {
  id = "/subscriptions/f26494cf-0ac8-4c97-
ne-dev-nsg/securityRules/allow-https-0"
  name = "allow-https-0"
  ~ source_address_prefix = "10.0.0.0/24" -> "192.168.1.0/24"
  # (16 unchanged attributes hidden)
}
```

The second change is changing rule[1] which was referencing 10.1.0.0/24 but will now reference 10.0.0.0/24 as its now second on the list, with a priority of 201...

```
# azure_rm_network_security_rule.allow_https[1] will be updated in-place
~ resource "azure_rm_network_security_rule" "allow_https" {
  id = "/subscriptions/f26494cf-0ac8-40
ne-dev-nsg/securityRules/allow-https-1"
  name = "allow-https-1"
  ~ source_address_prefix = "10.1.0.0/24" -> "10.0.0.0/24"
  # (16 unchanged attributes hidden)
}
```

The third change is changing rule[2] which was referencing 10.2.0.0/24 but will now reference 10.1.0.0/24 as this is now third on the list, with a priority of 202...

```
# azure_rm_network_security_rule.allow_https[2] will be updated in-place
~ resource "azure_rm_network_security_rule" "allow_https" {
  id = "/subscriptions/f26494cf-0ac8-40
ne-dev-nsg/securityRules/allow-https-2"
  name = "allow-https-2"
  ~ source_address_prefix = "10.2.0.0/24" -> "10.1.0.0/24"
  # (16 unchanged attributes hidden)
}
```

That leaves 10.2.0.0/24, which is now fourth in the list, and this is the '1 to add'. Rule[3] will be created and reference 10.2.0.0/24 with a priority of 203...

```
# azure_rm_network_security_rule.allow_https[3] will be created
+ resource "azure_rm_network_security_rule" "allow_https" {
+   access = "Allow"
+   destination_address_prefix = "*"
+   destination_port_ranges = [
+     "443",
+   ]
+   direction = "Inbound"
+   id = (known after apply)
+   name = "allow-https-3"
+   network_security_group_name = "landing-zone-dev-nsg"
+   priority = 203
+   protocol = "Tcp"
+   resource_group_name = "landing-zone-dev-rg"
+   source_address_prefix = "10.2.0.0/24"
+   source_port_range = "*"
+ }
Plan: 1 to add, 3 to change, 0 to destroy.
```

14. Apply the changes

terraform apply --auto-approve

Priority ↑↓	Name ↑↓	Port ↑↓	Protocol ↑↓	Source ↑↓
▽ Inbound Security Rules				
200	allow-https-0	443	Tcp	192.168.1.0/24
201	allow-https-1	443	Tcp	10.0.0.0/24
202	allow-https-2	443	Tcp	10.1.0.0/24
203	allow-https-3	443	Tcp	10.2.0.0/24

With count, resource identity is bound to list position. Any insertions or removals shift the indexes, forcing Terraform to update values (such as CIDR or priority) even

when the intent has not changed. This is **configuration churn** - unnecessary updates caused by order dependence.

Phase 4 – Identity churn

15. Comment line **20** and uncomment line **21** of **main.tf**

This will change the name suffix given to the rules from a simple index digit to the cidr itself, sanitized to satisfy naming syntax requirements. So, the rule name that was 'allow-https-0' will become 'allow-https-192-168-1-0-24'

16. Destroy all current rules for a clean start;

terraform destroy --auto-approve

17. Apply the new configuration to create rules using the new naming convention. Verify in state and in the portal

terraform apply --auto-approve

terraform state list

*azurerm_network_security_group.app
azurerm_network_security_rule.allow_https[0]
azurerm_network_security_rule.allow_https[1]
azurerm_network_security_rule.allow_https[2]
azurerm_network_security_rule.allow_https[3]
azurerm_resource_group.rg*

Priority ↑↓	Name ↑↓	Port ↑↓	Protocol ↑↓	Source ↑↓
✓ Inbound Security Rules				
200	allow-https-192-168-1-0-24	443	Tcp	192.168.1.0/24
201	allow-https-10-0-0-0-24	443	Tcp	10.0.0.0/24
202	allow-https-10-1-0-0-24	443	Tcp	10.1.0.0/24
203	allow-https-10-2-0-0-24	443	Tcp	10.2.0.0/24

18. Now we have established the new resource naming convention, let us see how re-indexing affects things. Comment line **8** and uncomment line **9** of terraform.tfvars (a cidr has been removed from the list)

Plan the changes

terraform plan

The plan is 3 to add, 4 to destroy.

How can one deletion merit the destruction of all of the existing resources followed by the creation of three new ones ?

Scroll up and review the plan.

Before the proposed change, the position of the cidr in the list determined the name and priority of the resource:

```
azurerm_network_security_rule.allow_https[0] > "allow-https-192-168-1-0-24" 200
azurerm_network_security_rule.allow_https[1] > "allow-https-10-0-0-0-24" 201
azurerm_network_security_rule.allow_https[2] > "allow-https-10-1-0-0-24" 202
azurerm_network_security_rule.allow_https[3] > "allow-https-10-2-0-0-24" 203
```

Based on the new list, we are asking that the names of the resources be changed. The priorities will also be changed to reflect the index position:

```
azurerm_network_security_rule.allow_https[0] > "allow-https-10-0-0-0-24" 200
azurerm_network_security_rule.allow_https[1] > "allow-https-10-1-0-0-24" 201
azurerm_network_security_rule.allow_https[2] > "allow-https-10-2-0-0-24" 202
azurerm_network_security_rule.allow_https[3] - no longer needed
```

Changing the name of a resource is a **ForceNew** change, meaning that the original resource must be destroyed and a new one created. All four resources are therefore deleted; three because the rule names of [0],[1] and [2] are to be changed and [3] because there are now only 3 elements in the list, so this fourth one is not needed. Having destroyed all the resources, 3 new ones are created with the new rule names; [0] allow-https-10-0-0-0-24, [1] allow-https-10-1-0-0-24 and [2] allow-https-10-2-0-0-24"

This is identity churn !

19. Apply the changes and verify;

terraform apply --parallelism=1 --auto-approve

Priority ↑↓	Name ↑↓	Port ↑↓	Protocol ↑↓	Source ↑↓
✓ Inbound Security Rules				
200	allow-https-10-0-0-0-24	443	Tcp	10.0.0.0/24
201	allow-https-10-1-0-0-24	443	Tcp	10.1.0.0/24
202	allow-https-10-2-0-0-24	443	Tcp	10.2.0.0/24

* Terraform applies resources in parallel by default to speed up execution. While efficient, this can cause race conditions when resources have ordering dependencies that Terraform cannot fully infer, particularly during imports, migrations, or destructive changes. Using: **terraform apply --parallelism=1** forces Terraform to apply changes sequentially. This reduces the risk of transient failures by ensuring resources are created, modified, or destroyed in a predictable order. In high-risk or governance-sensitive scenarios, serial execution trades speed for reliability and clearer operational behaviour.

Deriving names or other ForceNew fields from volatile data (like CIDR order) can cause true replacements. When those values change, Terraform must destroy and recreate the resources. This demonstrates **identity churn** - replacements caused by identity change, not intent change.

Phase 5 - For_each to the rescue ?

20. Destroy all existing rules before shifting to 'for-each' option;

terraform destroy -target="azurerm_network_security_rule.allow_https"

* Terraform allows selective operations using the -target flag, as used here. This instructs Terraform to act on a specific resource rather than the full configuration. While useful for recovering from failed deployments or removing a known problematic resource, targeted operations bypass Terraform's normal holistic planning. Dependencies may be implicitly affected, and the resulting state may no longer reflect the intended architecture described in code.

For this reason, -target should be treated as an **exceptional recovery tool**, not a routine workflow. In governed environments, targeted destruction increases the risk of configuration drift and should be followed by a full terraform plan to verify that the overall state remains consistent and compliant.

21. Enter **'yes'** to confirm the destroy

22. In **main.tf**, Deactivate the current rules workflow block that is using count (**see line 15**) and activate a replacement workflow block that will use for_each (**see line 40**)

This resource block switches from count to for_each methodology. Terraform no longer creates resources addressed purely by index values, [0], [1], now it will create them using the cidr itself as the key. The priority of every NSG rule must be unique, and, as it is not specified, we are forced to use the list position as an index (lines 44-47);

```
for_each = { for idx, cidr in var.allowed_cidrs : cidr => 200 + idx }
```

This creates the dynamic map;

KEY	VALUE
"10.0.0.0/24"	200
"10.1.0.0/24"	201
"10.2.0.0/24"	202

23. Apply the changes

terraform apply --auto-approve

24. Use the portal and state list to verify the resources are created using the new naming convention and that state uses the cidr in its addressing..

Priority ↑↓	Name ↑↓	Port ↑↓	Protocol ↑↓	Source ↑↓
✓ Inbound Security Rules				
200	allow-https-10_0_0_24	443	Tcp	10.0.0.0/24
201	allow-https-10_1_0_24	443	Tcp	10.1.0.0/24
202	allow-https-10_2_0_24	443	Tcp	10.2.0.0/24

terraform state list

```
azurerm_network_security_group.app
azurerm_network_security_rule.allow_https["10.0.0.0/24"]
azurerm_network_security_rule.allow_https["10.1.0.0/24"]
azurerm_network_security_rule.allow_https["10.2.0.0/24"]
azurerm_resource_group.rg
```

Switching to **for_each** replaces numeric indexes with meaningful keys (CIDRs). Resource addresses are now stable, even when list order changes. We have solved identity churn, but rule priorities still depend on order—so configuration churn can still occur.

25. Comment line **9** and uncomment line **10** of terraform.tfvars (a new cidr has been inserted at start of the list)

26. Plan the changes

terraform plan

The plan is 1 to add, 3 to change

Looking at the plan output identifies the reason for the 3 proposed changes, the change in priorities that have arisen due to the index shift when inserting a new element into the list. The new resource added is the new cidr entry and its priority is purely based on where it is positioned in the list.

This is more configuration churn !

27. Apply the changes

terraform apply --parallelism=1 --auto-approve

Note: Even with parallelism set at 1 there can still be issues with order reconfiguration. You will have to repeat the above command twice before all resources are eventually reconfigured.

Not explicitly defining the rule priority, allowing the position of the cidr in the list to be its determinant, will potentially cause configuration churn whether we use count or for_each. But not using the list position as a naming determinant will always prevent identity churn when we use for_each.

Even with for_each, derived attributes like 'priority = 200 + idx' still shift when items move. This can produce configuration churn. To eliminate it, decouple rule attributes from position altogether and make them explicit.

Phase 6 - Maps to the rescue !

28. Destroy all existing resources before shifting to map option;

terraform destroy --auto-approve

29. Comment line **6** and uncomment line **7** of **variables.tf** (redefining the allow_cidrs variable as type map(number))

30. Comment line **10** and uncomment line **11** of **terraform.tfvars** (initial cidr-priority map values)

31. In **main.tf**, Deactivate the current rules workflow block that is using for_each and indexing (see line **40**) and activate a replacement workflow block that will use for_each without indexing (see line **71**)

This rules resource block still uses the for_each methodology but without having to rely on index position to determine the rule priority. The variable now has an explicit key; the cidr, and a value; the rule priority. The position of the cidr/priority in the variable values declaration is no longer significant in how terraform addresses the resource.

32. Apply the changes

terraform apply --auto-approve

Priority ↑↓	Name ↑↓	Port ↑↓	Protocol ↑↓	Source ↑↓
▽ Inbound Security Rules				
200	allow-https-10_1_0_0_24	443	Tcp	10.1.0.0/24
201	allow-https-192_168_1_0_...	443	Tcp	192.168.1.0/24
202	allow-https-10_2_0_0_24	443	Tcp	10.2.0.0/24

33. Comment line **11** and uncomment line **12** of **terraform.tfvars** (new entry and order shuffle)

34. Plan the changes

terraform plan

Plan: 1 to add, 0 to change, 0 to destroy

The re-ordering of the cidr/priority entries no longer affects the address terraform assigns to the resource, so no more churn.

35. Prove this assertion by manipulating the allowed cidr variable value entries. Add new cidr/priority combinations, remove existing one, shuffle the order, whatever takes your fancy. No configuration or identity churn should occur. **Note:** Edits, additions, and deletions are **not** churn; they are desired configuration changes.

Using a map (CIDR → priority) makes both identity and configuration fully deterministic. Keys provide stable addressing; explicit values prevent drift. The result: zero-diff plans for reorders, no unintended updates, and a perfect foundation for modular automation.

Phase 7 – Clean up that mess before you leave !

To prepare for the next teaching module we will destroy everything created here. Your work has not been in vain though as the lessons learnt here and the ‘good’ code generated will appear again in the next module, along with new additional code that need some attention.

Destroy all your current resources : **terraform destroy --auto-approve**

Congratulations, you have completed this lab

© QA 2026

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.